```
// firebase.js — интеграция Firestore + Auth для риск-калькулятора
// Firebase v11 modular SDK (загружается с CDN, работает в обычном <script type="module">)

import { initializeApp } from "https://www.gstatic.com/firebasejs/11.0.2/firebase-app.js";
import {
  getAuth, signInAnonymously, onAuthStateChanged
} from "https://www.gstatic.com/firebasejs/11.0.2/firebase-auth.js";
import {
  getFirestore, collection, doc,
  addDoc, setDoc, getDoc, getDocs, updateDoc, deleteDoc,
  query, orderBy, serverTimestamp
} from "https://www.gstatic.com/firebasejs/11.0.2/firebase-firestore.js";

// 1) Вставь сюда конфиг из консоли Firebase:
//    Project settings → General → Your apps → SDK setup and configuration
const firebaseConfig = {
```

```
  apiKey:            "ТВОЙ_API_KEY",
  authDomain:
"ТВОЙ_PROJECT.firebaseapp.com",
  projectId:         "ТВОЙ_PROJECT_ID",
  storageBucket:
"ТВОЙ_PROJECT.appspot.com",
  messagingSenderId: "ТВОЙ_SENDER_ID",
  appId:             "ТВОЙ_APP_ID"
};

const app  =
initializeApp(firebaseConfig);
const auth = getAuth(app);
const db   = getFirestore(app);

let uid = null;

// ---- Авторизация ----
// Анонимный вход = у каждого устройства
свой изолированный аккаунт без пароля.
// Позже можно заменить на вход по email
— данные привяжутся к uid так же.
export function initAuth(onReady) {
  onAuthStateChanged(auth, (user) => {
    if (user) { uid = user.uid; onReady
&& onReady(uid); }
  });
  signInAnonymously(auth).catch(err =>
console.error("Auth error:", err));
}

// Пути к коллекциям текущего
```

```
пользователя
const tradesCol = () => collection(db,
"users", uid, "trades");
const levelsCol = () => collection(db,
"users", uid, "levels");
const userDoc   = () => doc(db, "users",
uid);

// ---- Настройки ----
export async function
saveSettings(settings) {
  await setDoc(userDoc(), { ...settings,
updatedAt: serverTimestamp() }, { merge:
true });
}
export async function getSettings() {
  const snap = await getDoc(userDoc());
  return snap.exists() ? snap.data() :
null;
}

// ---- Сделки (история расчётов) ----
export async function saveTrade(trade) {
  const ref = await addDoc(tradesCol(), {
    pair:                trade.pair ??
"BTC/USDT",
    direction:           trade.direction,
    account:             trade.account,
    riskPercent:
trade.riskPercent,
    entryPrice:
trade.entryPrice,
```

```
    stopPrice:           trade.stopPrice,
    targetPrice:
trade.targetPrice ?? null,
    currentPrice:
trade.currentPrice,
    riskAmount:
trade.riskAmount,
    positionValue:
trade.positionValue,
    units:               trade.units,
    stopDistancePercent:
trade.stopDistancePercent,
    riskReward:          trade.riskReward
?? null,
    note:                trade.note ??
"",
    outcome:             "open",
    resultAmount:        null,
    createdAt:
serverTimestamp()
  });
  return ref.id;
}

export async function getTrades() {
  const q = query(tradesCol(),
orderBy("createdAt", "desc"));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id,
...d.data() }));
}
```

```
// Отметить исход сделки: outcome = "win"
| "loss" | "cancelled"
export async function
updateTradeOutcome(tradeId, outcome,
resultAmount = null) {
  await updateDoc(doc(tradesCol(),
tradeId), { outcome, resultAmount });
}

export async function
deleteTrade(tradeId) {
  await deleteDoc(doc(tradesCol(),
tradeId));
}

// ---- Уровни поддержки/сопротивления --
--
export async function saveLevel(level) {
  const ref = await addDoc(levelsCol(), {
    pair:      level.pair ?? "BTC/USDT",
    price:     level.price,
    type:      level.type,          //
"support" | "resistance"
    note:      level.note ?? "",
    createdAt: serverTimestamp()
  });
  return ref.id;
}
export async function getLevels() {
  const q = query(levelsCol(),
orderBy("price", "desc"));
  const snap = await getDocs(q);
```

```
  return snap.docs.map(d => ({ id: d.id,
...d.data() }));
}
export async function
deleteLevel(levelId) {
  await deleteDoc(doc(levelsCol(),
levelId));
}
```